

# Smart Backup

Sistema de Backup Inteligente — Especificação Completa do Sistema



17 de maio de 2026



Python 3.13+



11 módulos



~6.500+ linhas



## Sumário

1. Visão Geral
2. Stack Tecnológica
3. Estrutura do Projeto
4. Arquitetura e Fluxo
5. Funcionalidades
6. Modelo de Dados
7. Instalação e Execução
8. Variáveis de Ambiente
9. Testes
10. Funcionalidades Futuras
11. Padrões e Convenções
12. Resumo dos Módulos

# 01 Visão Geral

Propósito, escopo e objetivos do sistema

O **Smart Backup** é uma aplicação desktop desenvolvida em Python para **monitoramento contínuo** de diretórios, **classificação inteligente** de arquivos e **backup incremental** com deduplicação por hash, snapshots JSON e criptografia em envelope.

O sistema combina **monitoramento em tempo real** (watchdog), **análise de metadados** via árvore de decisão local com integração opcional à **Gemini API** (Google), e **armazenamento incremental** com restauração por manifesto. A interface gráfica em Tkinter oferece controle completo sobre backups, agendamento, histórico e restauração.



## 02 Stack Tecnológica

Linguagens, frameworks e bibliotecas utilizadas

|                                                                                                                                                                                |                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div> <b>Python 3.13+</b></div> <div>Runtime principal da aplicação</div>                     | <div> <b>Tkinter</b></div> <div>Interface desktop nativa do Python</div>                     |
| <div> <b>watchdog 6.0+</b></div> <div>Monitoramento de arquivos em tempo real</div>           | <div> <b>pandas 3.0+</b></div> <div>Manipulação de dados e geração de dataset</div>          |
| <div> <b>cryptography 46+</b></div> <div>AES-256-GCM, PBKDF2-SHA256, envelope de chaves</div> | <div> <b>pystray 0.19+</b></div> <div>Ícone na bandeja do sistema Windows</div>              |
| <div> <b>pillow 12+</b></div> <div>Manipulação de imagens e ícones</div>                    | <div> <b>Gemini API</b></div> <div>Classificação opcional via LLM (metadados apenas)</div> |

**Dependências completas:** customtkinter, cryptography, darkdetect, flake8, numpy, pandas, pillow, pystray, python-dateutil, watchdog

## 03 Estrutura do Projeto

Organização de diretórios e arquivos

```
📁 sistema-backup-inteligente/
├── main.py # Ponto de entrada
├── README.md
├── requirements.txt
├── .env.example
├──
├── 📁 auth/ # Autenticação e permissões
│   ├── __init__.py
│   ├── permissions.py
│   └── users.py
├──
├── 📁 backup/ # Núcleo do backup incremental
│   └── backup_manager.py # ~2500 linhas
├──
├── 📁 config/ # Configurações e estado
│   ├── config.json
│   ├── users.json
│   ├── backup_history.json
│   ├── backup_schedule.json
│   ├── file_activity.json
│   ├── llm_classification_cache.json
│   └── priority_backup_state.json
├──
├── 📁 backups/ # Armazenamento de backups
│   ├── 📁 admin/
│   │   ├── index.json
│   │   └── 📁 2026-05-12/
│   │       ├── 📁 arquivos_relacionados/
│   │       └── 📁 snapshots/
│   ├── 📁 dudu/
│   └── 📁 sistema/
├──
├── 📁 dataset/ # CSV com metadados
├── 📁 docs/ # Documentação
├── 📁 interface/ # GUI Tkinter
├── 📁 ml/ # Classificação inteligente
├── 📁 monitor/ # Watchdog
├── 📁 scanner/ # Varredura de diretórios
├── 📁 scheduler/ # Agendamento
├── 📁 security/ # Criptografia
```

- └─  tests/ # Testes
- └─  tray/ # Bandeja do sistema
- └─  utils/ # Utilitários

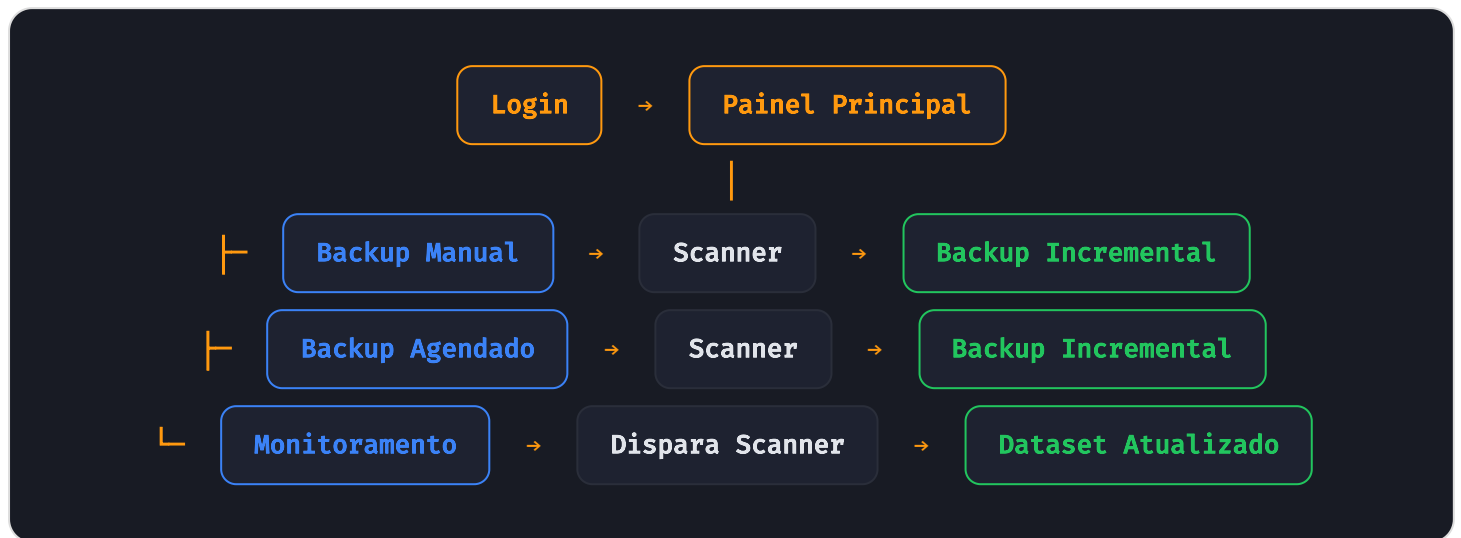
## 04 Arquitetura e Fluxo

Como os componentes se integram e o fluxo de execução

### 4.1 Inicialização

1. Garante pastas necessárias ( `config/` , `dataset/` , `ml/` )
2. Detecta se é primeira execução (config sem diretórios OU nenhum usuário)
3. Inicia **monitoramento** em thread separada (daemon)
4. Inicia **agendador** em thread separada (daemon)
5. Abre **tela de login/panel** na thread principal do Tkinter
6. Após fechar o painel, inicia o **ícone da bandeja**

### 4.2 Fluxo Geral



### 4.3 Fluxo de Backup Incremental



## 05 Funcionalidades Detalhadas

Descrição completa de cada funcionalidade implementada

### 5.1 Autenticação e Controle de Acesso AUTH/

#### Sistema de Usuários

- Armazenamento:** `config/users.json` com senhas hashadas via PBKDF2-SHA256 (260.000 iterações)
- Campos:** username, name, role, password (hash + salt), company\_id, metadados criptográficos
- Operações:** criar, autenticar, listar, atualizar, deletar, alterar senha
- Recuperação:** chave única formato `RECOVERY-XXXX-XXXX-XXXX-XXXX`
- Autenticação:** PBKDF2 + HMAC compare\_digest + descriptografia da chave mestra

#### Sistema de Permissões (RBAC)

| PERMISSÃO              | ADMIN | OPERADOR | VISUALIZADOR |
|------------------------|-------|----------|--------------|
| Gerenciar usuários     | ✓     | ✗        | ✗            |
| Gerenciar diretórios   | ✓     | ✗        | ✗            |
| Alterar destino backup | ✓     | ✗        | ✗            |
| Executar backup        | ✓     | ✓        | ✗            |
| Agendar backup         | ✓     | ✓        | ✗            |
| Ver arquivos           | ✓     | ✓        | ✓            |
| Ver histórico          | ✓     | ✓        | ✓            |
| Baixar backup          | ✓     | ✓        | ✗            |
| Restaurar backup       | ✓     | ✓        | ✗            |

**Visibilidade do histórico:** Admin vê todos os backups; Operador vê os próprios + de visualizadores; Visualizador vê apenas os próprios.

### 5.2 Monitoramento em Tempo Real MONITOR/

- Usa `watchdog.Observer` para monitorar diretórios configurados

- Eventos capturados: **criação, modificação, remoção e movimentação** de arquivos
- **Cooldown**: 2 segundos entre varreduras para evitar disparos excessivos
- Ignora caminhos internos: `config/` , `dataset/` , `.git/` , `__pycache__/`
- Dispara `run_scanner()` automaticamente ao detectar mudanças

5.3 Scanner e Dataset

SCANNER/

- Varre todos os diretórios configurados recursivamente
- Para cada arquivo coleta: nome, extensão, tipo, tamanho, datas, hash SHA-256, contadores de atividade, contexto do diretório
- Anota **arquivos duplicados** por hash
- Aplica **classificação** (regras + LLM opcional)
- Gera/atualiza `dataset/files_dataset.csv` via pandas

5.4 Classificação Inteligente

ML/

Árvore de Decisão Local

| CRITÉRIO                                             | IMPACTO | MOTIVO                               |
|------------------------------------------------------|---------|--------------------------------------|
| Modificações frequentes ( $\geq 2$ ou $\leq 2$ dias) | +18     | Arquivo modificado recentemente      |
| Extensão de alto valor                               | +18     | Extensão com valor para backup       |
| Extensão de sistema/programa                         | +20     | Relevante para sistema               |
| Acessos frequentes ( $\geq 2$ )                      | +14     | Arquivo acessado várias vezes        |
| Nome importante                                      | +24     | Indica documento, projeto, etc.      |
| Contexto importante                                  | +18     | Diretório indica contexto importante |
| Palavra-chave (scanner)                              | +10     | Keyword detectada                    |
| Conteúdo duplicado                                   | -6      | Hash duplicado encontrado            |
| Extensão de baixo valor                              | -18     | Temporário/descartável               |
| Contexto de baixo valor                              | -35     | Cache, venv, temp                    |

Limiares:  $\geq 70 \rightarrow$  **alta**,  $\geq 40 \rightarrow$  **média**,  $< 40 \rightarrow$  **baixa**

Integração Gemini API (opcional)

- Ativada via `GEMINI_API_KEY` (variável de ambiente ou `.env` )
- Envia apenas **metadados** — nunca o conteúdo do arquivo
- Prompt estruturado pedindo classificação JSON
- Cache local (até 1000 entradas) para evitar chamadas repetidas
- Fallback automático para regras locais se API falhar
- Modelo padrão: `gemini-2.5-flash`
- Mescla resultado da LLM com regras locais (prioridade mais alta vence)

5.5 Backup Incremental com Deduplicação BACKUP/

Estrutura de Armazenamento

```
backups/<usuario>/
├── index.json
├── YYYY-MM-DD/
│   ├── arquivos_relacionados/
│   │   └── <hash_sha256>
│   └── snapshots/
│       └── snapshot_2026-05-12_08-00-00.json
```

Mecanismo de Deduplicação

- Cada arquivo é armazenado **uma única vez**, identificado por hash SHA-256
- Se o hash já existe, apenas uma **referência** é criada
- O `index.json` mantém contagem de referências para cada objeto

Status dos Arquivos

| STATUS                                  | SIGNIFICADO                                |
|-----------------------------------------|--------------------------------------------|
| <code>stored_new_object</code>          | Novo objeto armazenado                     |
| <code>referenced_existing_object</code> | Hash já existente, apenas referência       |
| <code>skipped_unchanged</code>          | Arquivo não modificado desde último backup |
| <code>skipped_not_eligible</code>       | Não elegível pela política de prioridade   |
| <code>error</code>                      | Falha ao processar                         |

Política de Backup por Prioridade

| PRIORIDADE | PRODUÇÃO                  | DEV MODE      |
|------------|---------------------------|---------------|
| baixa      | A cada 7 dias             | A cada 30 min |
| média      | A cada 2 dias             | A cada 15 min |
| alta       | Início do dia + a cada 4h | A cada 5 min  |

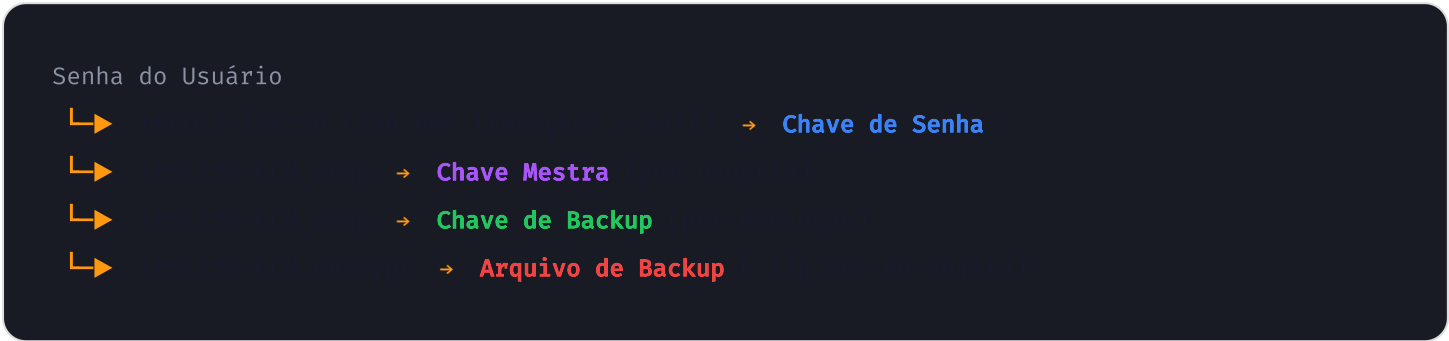
Verificação a cada 10 min (produção) ou 1 min (DEV MODE). Estado persistente em `config/priority_backup_state.json` .

Agendamento

- Thread separada com loop a cada 20s
- Verifica se há backup agendado pendente via janela de horário
- Processa backup por prioridade se habilitado
- Suporta horário de início e fim para agendamento diário

5.6 Criptografia SECURITY/

Esquema de Criptografia em Envelope



- **Algoritmo:** AES-256-GCM com nonces únicos de 12 bytes
- **KDF:** PBKDF2-SHA256 com salt de 16 bytes
- **Tag de autenticação:** embutida no ciphertext
- **Troca de senha:** apenas a chave mestra é recriptografada; backups intactos
- **Redefinição sem senha antiga:** gera nova chave mestra → perde acesso a backups antigos

5.7 Restauração BACKUP/

- Restaura arquivos de snapshots incrementais
- Suporta estratégias de conflito: **renomear** ou **pular**

- Preserva datas de modificação originais
- Exporta snapshot completo para ZIP
- Busca por nome de arquivo na interface

### 5.8 Interface Gráfica

INTERFACE/

- **Tema escuro** profissional (paleta #283241, #1F2733, #FF990F)
- **Fontes:** Segoe UI (títulos: Segoe UI Black)
- Menu central com cards informativos e botões com profundidade
- Scrollbars estilizados e barra de progresso
- Cancelamento seguro de backup em segundo plano
- Rodapé com resumo do último backup

| TELA                 | FUNCIONALIDADES                                   |
|----------------------|---------------------------------------------------|
| Login                | Autenticação ou criação do primeiro administrador |
| Painel Principal     | Menu central, cards de estatísticas, resumo       |
| Gerenciar Diretórios | Adicionar/remover diretórios monitorados          |
| Executar Backup      | Nomear, descrever, executar com progresso         |
| Histórico            | Filtros por data, usuário, busca com sugestões    |
| Arquivos Analisados  | Status de cobertura, filtros por coluna           |
| Restauração          | Busca por nome, recuperação de versões            |
| Configurações        | Destino, agendamento, política de prioridade      |
| Gerenciar Usuários   | CRUD de usuários (admin apenas)                   |

### 5.9 Bandeja do Sistema

TRAY/

- Ícone na bandeja do Windows via **pystray**
- **Clique esquerdo:** abre o painel principal
- **Clique direito:** menu (Abrir painel, status do último backup, destino, Executar scan, Sair)
- Tooltip com informações resumidas (limitado a 120 caracteres)

## 06 Modelo de Dados

Estruturas de dados e formatos de arquivo

### 6.1 Configuração Principal ( `config/config.json` )

```
{
  "directories": ["C:/path/to/monitor"],
  "backup_destination": "C:/path/to/backups",
  "priority_backup_policy_enabled": true,
  "llm_classification_enabled": true,
  "llm_cache_enabled": true,
  "gemini_model": "gemini-2.5-flash",
  "deduplicate_backup": true
}
```

### 6.2 Dataset ( `dataset/files_dataset.csv` )

| COLUNA                             | DESCRIÇÃO                                                  |
|------------------------------------|------------------------------------------------------------|
| <code>name</code>                  | Nome do arquivo                                            |
| <code>extension</code>             | Extensão (sem ponto)                                       |
| <code>type</code>                  | Tipo (document, code, image, spreadsheet, database, other) |
| <code>source_path</code>           | Caminho absoluto                                           |
| <code>archive_name</code>          | Caminho relativo no backup                                 |
| <code>size_kb</code>               | Tamanho em KB                                              |
| <code>file_hash</code>             | SHA-256 do conteúdo                                        |
| <code>priority</code>              | baixa / media / alta                                       |
| <code>priority_score</code>        | Pontuação 0-100                                            |
| <code>priority_reason</code>       | Motivos da decisão                                         |
| <code>classification_source</code> | rules / rules_fallback / gemini_api / gemini_cache         |
| <code>llm_confidence</code>        | Confiança 0.0-1.0                                          |
|                                    |                                                            |

| COLUNA        | DESCRIÇÃO                       |
|---------------|---------------------------------|
| backup_policy | Política derivada da prioridade |
| decision_tree | JSON das decisões da árvore     |
| is_duplicate  | Flag de arquivo duplicado       |
| important     | 1 se prioridade média ou alta   |

### 6.3 Snapshot Incremental ( `snapshot_*.json` )

```
{
  "snapshot_id": "snapshot_2026-05-12_22-26-57",
  "created_at": "2026-05-12T22:26:57",
  "trigger": "manual",
  "total_files": 28,
  "status_counts": { "stored_new_object": 28 },
  "files": [
    {
      "archive_name": "Pasta/arquivo.pdf",
      "file_name": "arquivo.pdf",
      "hash": "abc123 ...",
      "object_path": "2026-05-12/arquivos_relacionados/abc123 ...",
      "priority": "alta",
      "score": 85,
      "size": 6969,
      "status": "stored_new_object",
      "encryption": { ... }
    }
  ],
  "encryption": { "enabled": true, ... }
}
```

## 07 Instalação e Execução

Pré-requisitos e passos para execução

---

### Pré-requisitos

- Python 3.13 ou superior
- pip
- Ambiente Windows recomendado
- Chave `GEMINI_API_KEY` opcional para classificação com Gemini

### Passos

```
# 1. Criar ambiente virtual
python -m venv venv
venv\Scripts\activate

# 2. Instalar dependências
pip install -r requirements.txt

# 3. (Opcional) Configurar chave Gemini
$env:GEMINI_API_KEY = "sua-chave-aqui"

# 4. Executar
python main.py
```

## 08 Variáveis de Ambiente

Configuração via environment variables

| VARIÁVEL                | DESCRIÇÃO                                   | PADRÃO           |
|-------------------------|---------------------------------------------|------------------|
| GEMINI_API_KEY          | Chave da API Gemini                         | —                |
| GOOGLE_API_KEY          | Alternativa para Gemini                     | —                |
| GEMINI_MODEL            | Modelo Gemini                               | gemini-2.5-flash |
| SMARTBACKUP_LLM_ENABLED | Forçar ativação/desativação da LLM          | —                |
| BACKUP_DEV_MODE         | Modo desenvolvimento (intervalos reduzidos) | false            |

 **Importante:** Nunca salve a chave da API dentro do código. Use variáveis de ambiente ou arquivo `.env` (incluído no `.gitignore`).

## 09 Testes

Testes automatizados do sistema



### test\_encryption.py

Testes do sistema de criptografia em envelope (AES-256-GCM, PBKDF2)



### test\_file\_backup\_status.py

Testes de status de cobertura dos arquivos no backup



### test\_incremental\_backup.py

Testes do fluxo completo de backup incremental com deduplicação

## 10 Funcionalidades Futuras

Melhorias planejadas não implementadas

---



### Visualização de tamanho

Exibir tamanho dos backups e status da última execução no painel



### Agendamento avançado

Configuração mais granular de horários e dias do agendamento

## 11 Padrões e Convenções

Estilo de código, organização e boas práticas

---

- **Código:** PEP 8 (verificado com flake8)
- **Interface:** Tkinter puro com tema escuro personalizado
- **Nomes:** Português (BR) para variáveis, comentários e interface
- **Módulos:** Organização por funcionalidade (auth, backup, monitor, scanner, etc.)
- **Persistência:** JSON para configurações e estado; CSV para dataset
- **Threads:** Monitoramento e agendador em daemon threads; interface na thread principal
- **Segurança:** Hash de senha com PBKDF2; criptografia em envelope AES-256-GCM

## 12 Resumo dos Módulos

Responsabilidades e tamanho de cada módulo

| MÓDULO         | ARQUIVO(S)                   | RESPONSABILIDADE                                  | LINHAS  |
|----------------|------------------------------|---------------------------------------------------|---------|
| main           | main.py                      | Ponto de entrada, inicialização                   | ~50     |
| auth           | users.py ,<br>permissions.py | Autenticação, RBAC, gestão de usuários            | ~400    |
| backup         | backup_manager.py            | Backup incremental, dedup, restauração, histórico | ~2500   |
| scanner        | scanner.py                   | Varredura, geração de dataset, classificação      | ~400    |
| ml             | llm_classifier.py            | Classificação por regras + Gemini API             | ~850    |
| monitor        | monitor.py                   | Watchdog em tempo real                            | ~100    |
| scheduler      | scheduler.py                 | Agendamento em segundo plano                      | ~70     |
| interface      | gui.py , login.py            | Interface gráfica Tkinter (~1800+ linhas)         | ~2000+  |
| tray           | tray_icon.py                 | Ícone de bandeja do Windows                       | ~130    |
| security       | crypto_service.py            | Criptografia AES-256-GCM + envelope               | ~250    |
| utils          | file_hash.py                 | Cálculo de hash SHA-256                           | ~15     |
| tests          | 3 arquivos                   | Testes automatizados                              | —       |
| Total estimado |                              |                                                   | ~6.500+ |

